
BOOTCAMP MLOps

BOOTCAMP MLOps:

What we will learn to day:
Serving Models as a Live API

Danh Pham

Goals

- Answer the question: Why Do We Need an API?
- Learn about the tool to handle REST API in python - FastAPI.
- Know how to write APIs by yourself.

COLE

Contents

The Problem - Why Do We Need an API?

The Tools - FastAPI and Pydantic

The Blueprint for Our Lab

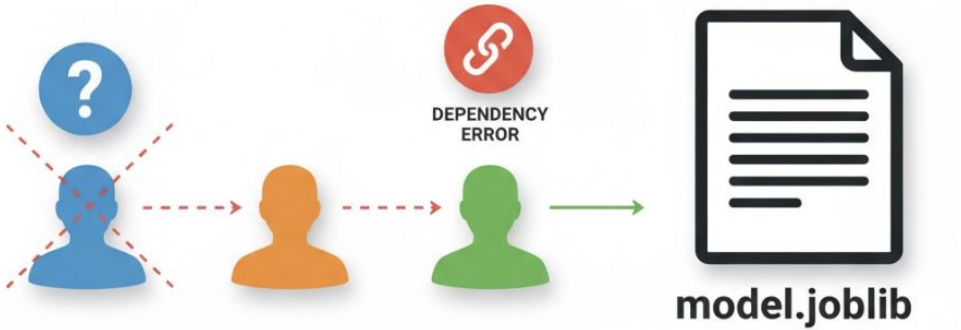
Hands-on Lab

COLE

Why Do We Need an API?

The Limits of a Static File

- Dependency Hell: Users need correct Python version, libraries, and hardware.
- No Control: Can't update model without redistributing.
- No Monitoring: No idea who uses model or its performance.

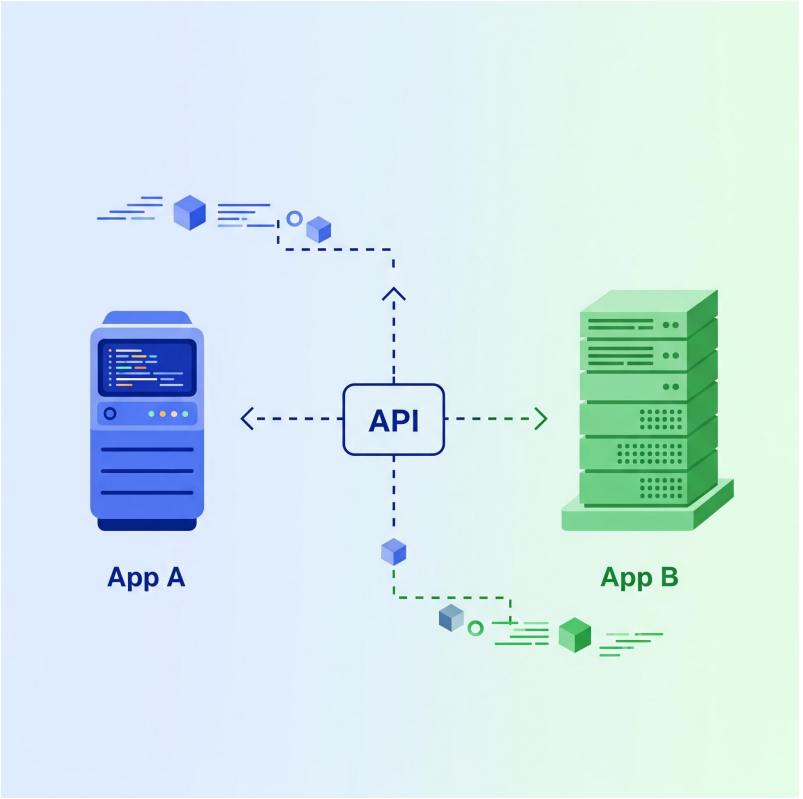


The Solution: Application Programming Interface

An API is a contract for application communication.

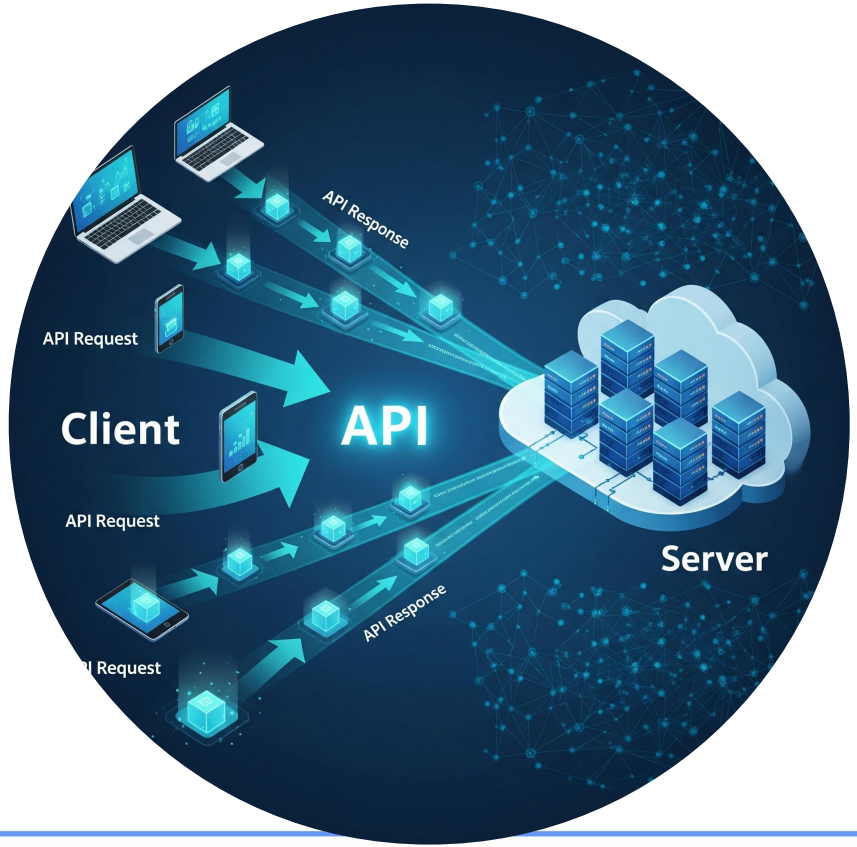
It hides complexity, simplifying user interaction.

Users send web requests, receiving predictions back.



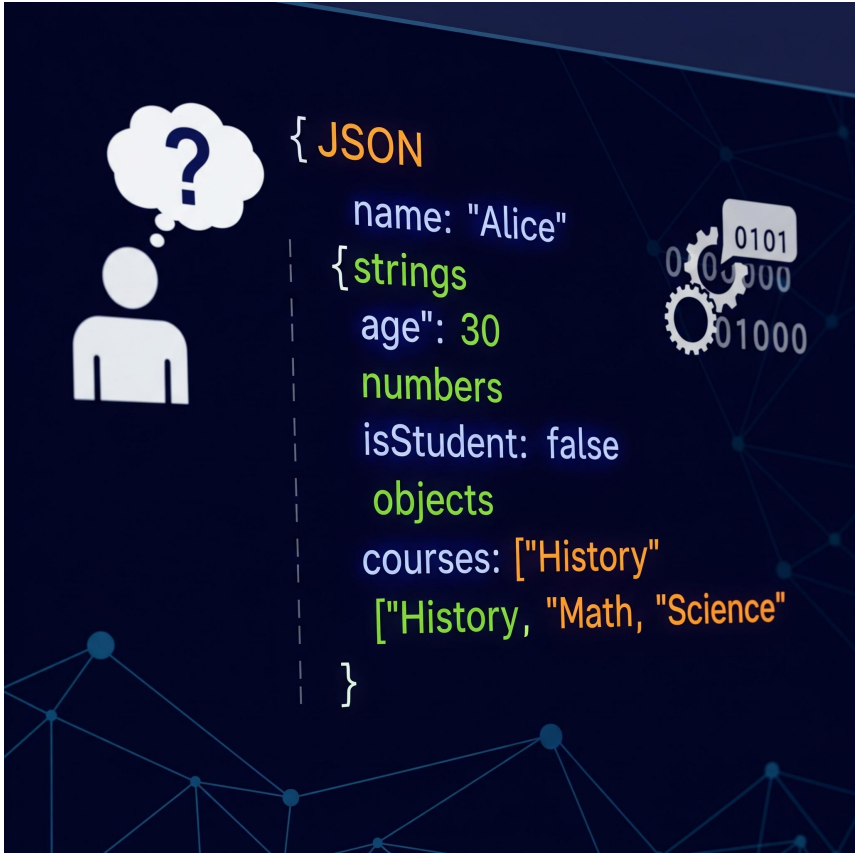
How APIs Work: The Client-Server Model

- APIs facilitate client-server communication
- Clients send requests to the server
- Server processes and responds to requests
- Communication happens via HTTP over the internet



The Language of APIs: JSON

- JSON is the standard for sending data.
- It is human-readable and machine-parseable.
- Key-value pairs structure JSON data.
- Example: { "square_feet": 1500 }

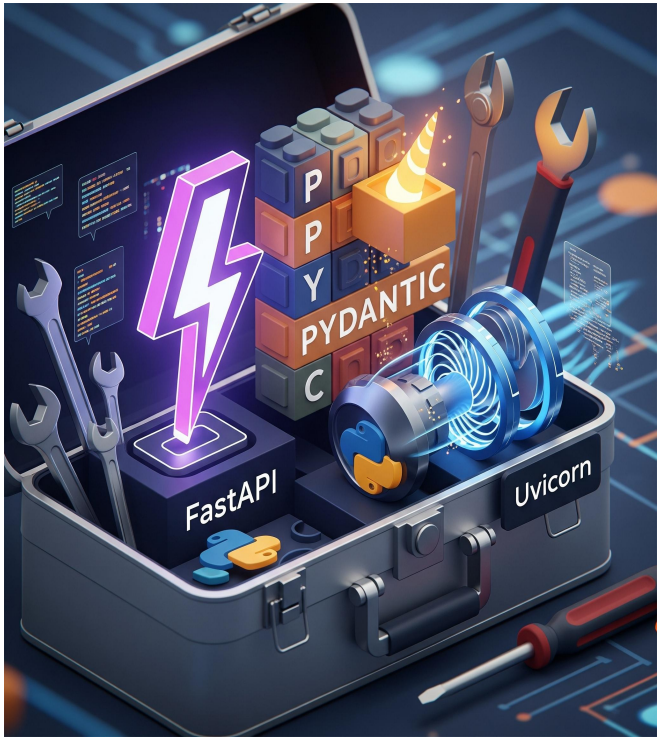


Our Toolkit for Building the API

FastAPI builds web API endpoints

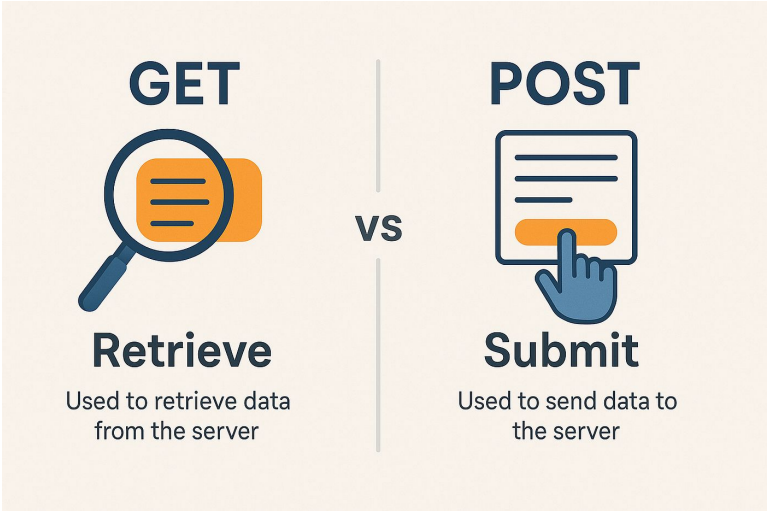
Pydantic validates the data

Uvicorn serves our application



API Verbs: GET vs. POST

- GET retrieves data from a server.
- POST sends data for creation or processing.
- We will use POST for sending prediction features.



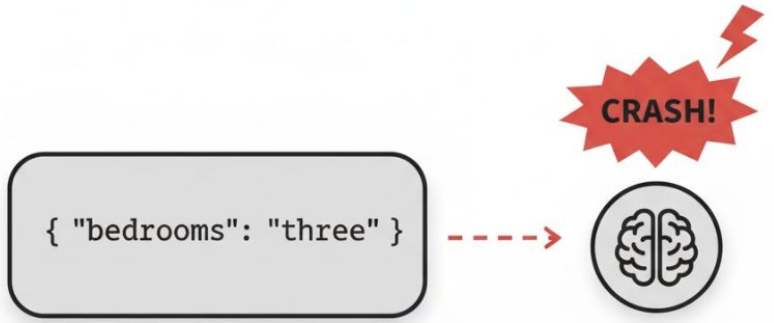
What is FastAPI?

- Modern, high-performance Python web framework
- Fast: One of the fastest Python frameworks available
- Easy: Designed for simple use and learning
- Automatic API documentation is generated



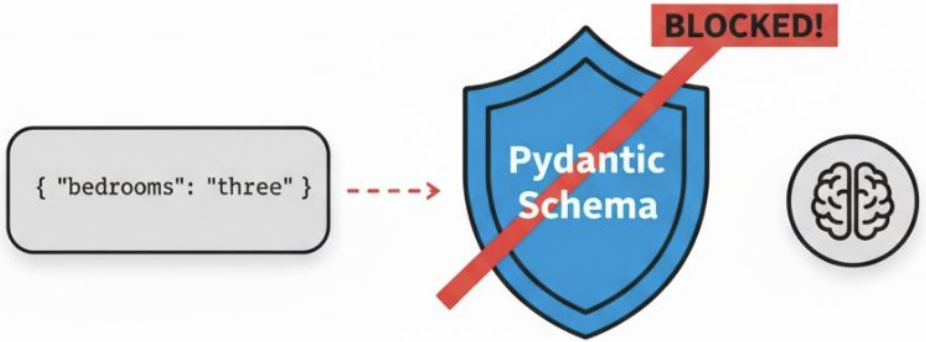
The Problem: What if the Client Sends Bad Data?

- Our model expects bedrooms to be a number (e.g., 3), not a word ("three").
- This would cause our code to crash. **Never trust client input!**



The Guardian: Pydantic for Data Validation

- Pydantic enforces a **schema** for our input data.
- It acts as a guard at the entrance of our API.
- If data is invalid, Pydantic automatically rejects it with a clear error message.



How Pydantic Works: A Simple Schema

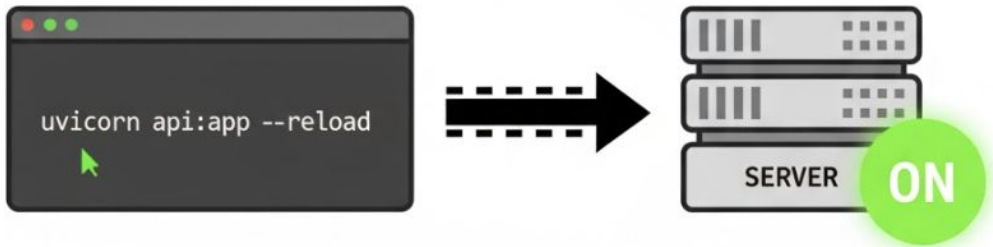
- We define a simple class that inherits from BaseModel.
- We use standard Python type hints (int, float, str).



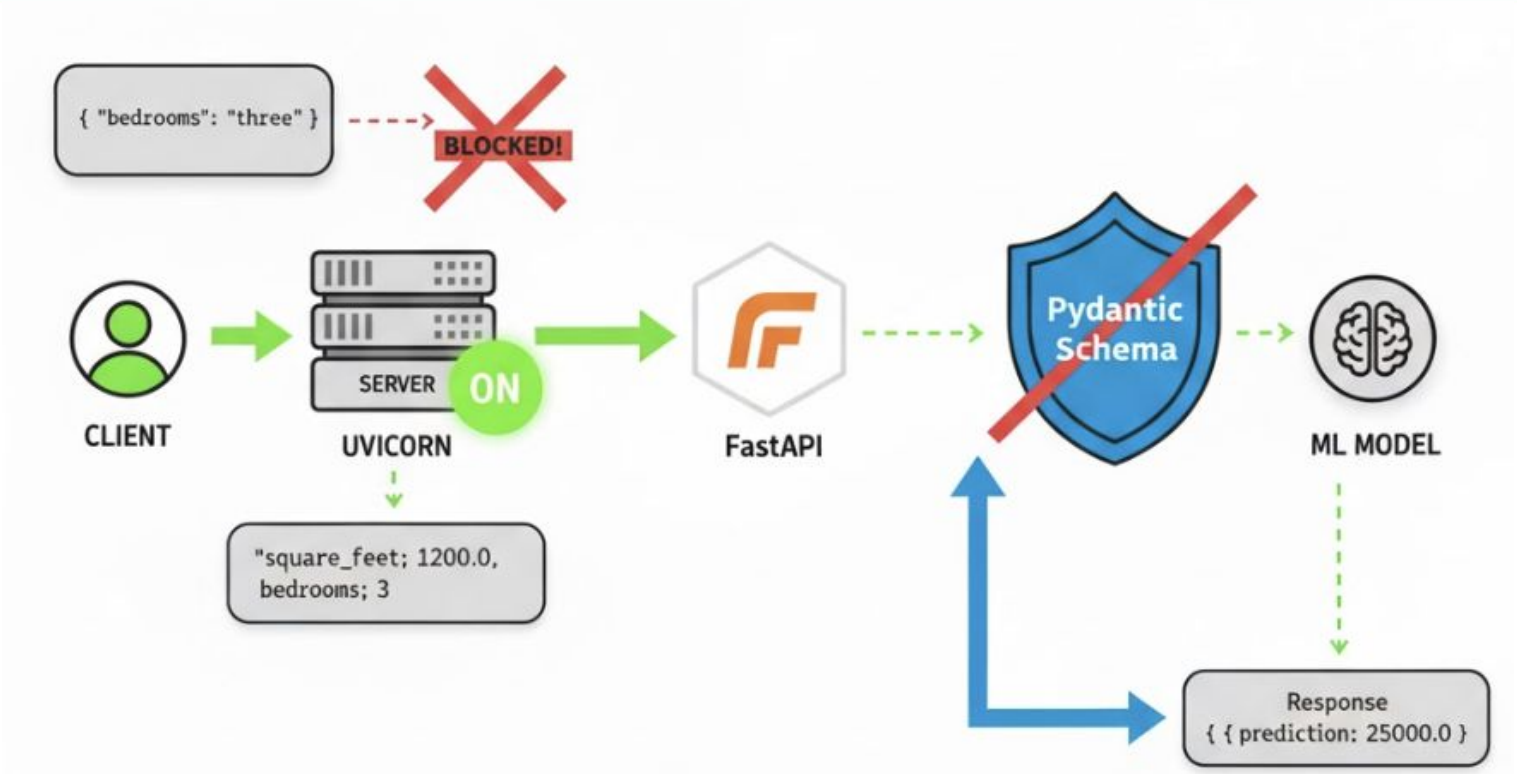
COLE

The Engine: Uvicorn

- Uvicorn is an ASGI server.
- It's the program that actually runs our FastAPI application.

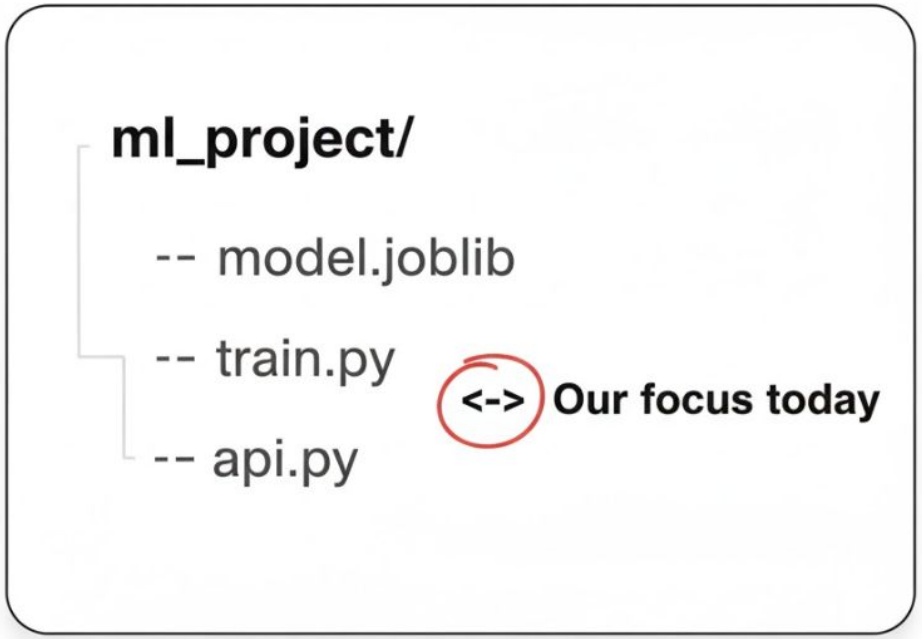


Recap: How It All Fits Together



Our Lab: Project Structure

- We will work in a new file: api.py.
- This file will contain all our API logic and load model.joblib.



Step 1: The FastAPI Boilerplate

- Import FastAPI and create an app instance.
- Define a root (/) endpoint to check if the server is running.

scripts > session_3 > api.py > Python > read_root

```python
from fastapi import FastAPI
from scripts.session_3.schemas.request import HousingPredictionRequest
from scripts.session_3.schemas.response import HousingPredictionResponse
import mlflow.sklearn
import pandas as pd
import uvicorn

app = FastAPI()

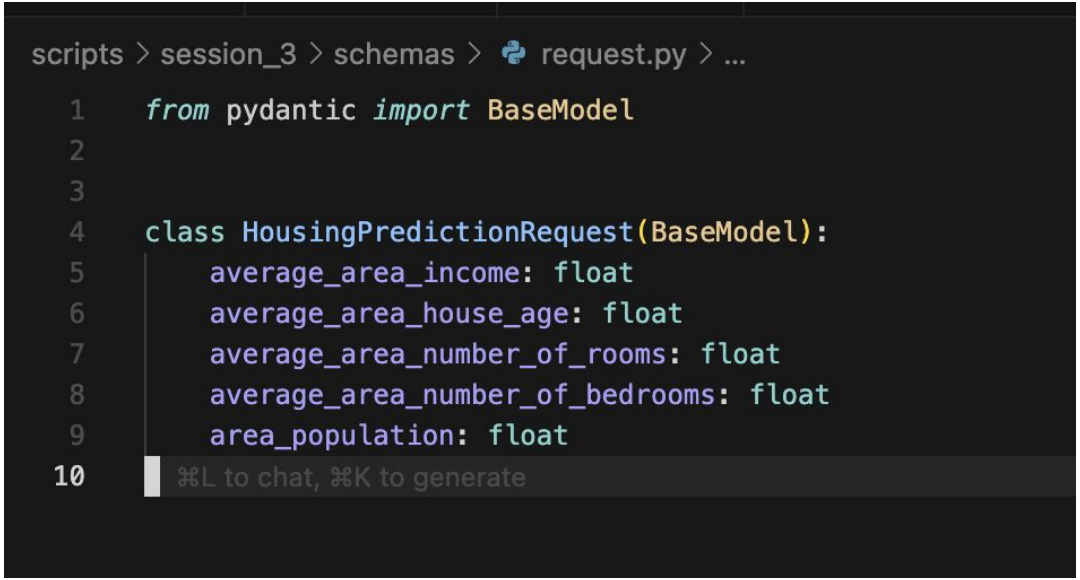



@app.get("/")
def read_root() -> dict[str, str]:
    return {"message": "Hello, FastAPI!"}

```

Step 2: Define the Input Schema with Pydantic

- Create a class inheriting from pydantic.BaseModel.
- Add attributes for every feature your model expects.



Step 3: The /predict Endpoint

- Use the @app.post("/predict") decorator.
- The function argument uses our Pydantic class as a type hint. FastAPI handles the validation.

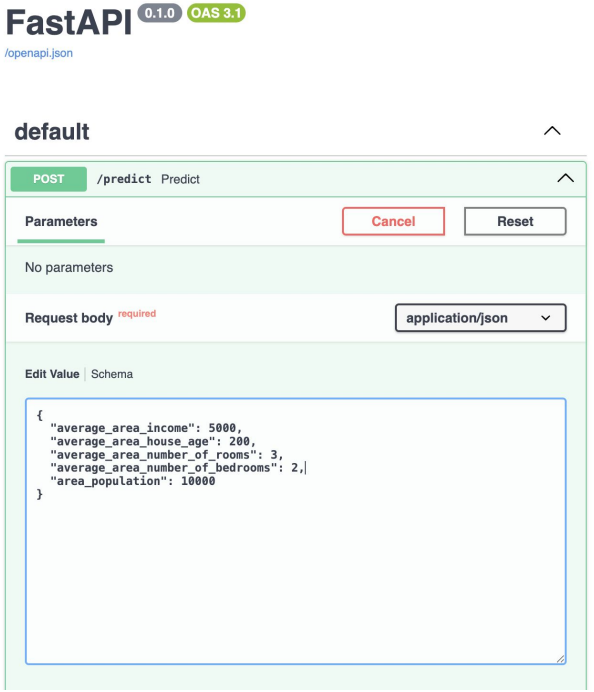
```python
model_name = "housing_prediction"
model_version = "1"
model_uri: LiteralString = f"models:/{model_name}/{model_version}"
model: Any | None = mlflow.sklearn.load_model(model_uri)

@app.post("/predict", response_model=HousingPredictionResponse)
def predict(request: HousingPredictionRequest) -> HousingPredictionResponse:
    input_data: dict[str, list[float]] = {
        "Avg. Area Income": [request.average_area_income],
        "Avg. Area House Age": [request.average_area_house_age],
        "Avg. Area Number of Rooms": [request.average_area_number_of_rooms],
        "Avg. Area Number of Bedrooms": [request.average_area_number_of_bedrooms],
        "Area Population": [request.area_population],
    }
    df = pd.DataFrame(input_data)
    predictions = model.predict(df)
    return HousingPredictionResponse(predicted_price=predictions[0])
```

COLE

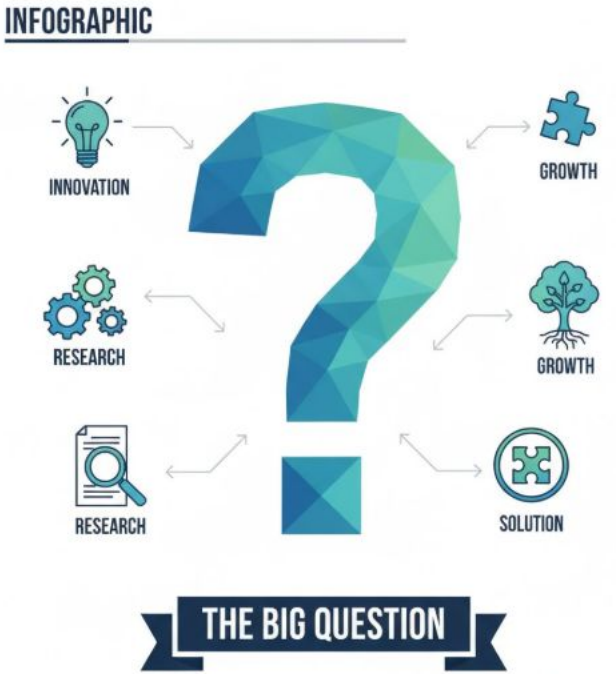
Step 4: Run with Uvicorn and Test

- Run from terminal: uvicorn api:app --reload
- Open browser to http://127.0.0.1:8000/docs
- Use the interactive UI to send a test prediction request.



Ready to Build?

- Any Questions Before We Begin?
- Let's start the lab!



What We Accomplished Today

- APIs are crucial for serving ML models effectively.
- FastAPI, Pydantic, and Uvicorn are key tools.
- A live prediction endpoint was built and tested.
- Pydantic provides robust data validation



For Next Time: Get Ready to Containerize!

1. Experiment with your API by sending invalid data and adding new features.
2. Read about Docker's core concepts.
3. Install Docker Desktop before the next session.



Docker + FASTAI Integration

COLE

Your mission

- **Goal:** Integrate MLflow into our train.py script.
- **Steps:**
  1. Install MLflow.
  2. Add logging code.
  3. Run multiple experiments.
  4. Compare results in the UI.
  5. Register our best model.

Q&A



Bài tập thực hành

Write API to make prediction for Iris model

Thank you for listening!

